Department of Computer Science,

CUI,Attock Campus.

Program: BSSE

Assignment no:3

Information Security (LAB)

Student Name	**Ahsan Rehman Khan**
Registration #	FA24-BSE-056

Professor: Ms. Ambreen Gul

Task 1:

Sol:

Input:

```
import hashlib
import datetime

# Block Class ----------------
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = str(datetime.datetime.now())
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        value = str(self.index) + self.timestamp + self.data + self.previous_hash
        return hashlib.sha256(value.encode()).hexdigest()

# Blockchain Class ----------------
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def add_block(self, data):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)
```

```
    def display_chain(self):
        for block in self.chain:
            print("\nIndex:", block.index)
            print("Timestamp:", block.timestamp)
            print("Data:", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Hash:", block.hash)

# Test ----------------
bc = Blockchain()

bc.add_block("Block 1 Data")
bc.add_block("Block 2 Data")
bc.add_block("Block 3 Data")
bc.add_block("Block 4 Data")

bc.display_chain()

# Modify Block ----------------
print("After Changing Block 2 Data :")
bc.chain[2].data = "Hacked Data"
bc.chain[2].hash = bc.chain[2].calculate_hash()

bc.display_chain()
```

Output:

```
Index: 0
Timestamp: 2026-05-04 08:44:31.158374
Data: Genesis Block
Previous Hash: 0
Hash: 98b43e28d1d88735e68435db6ba29d0df8c27edc1cdb33f61e07b0ac133f9c8f

Index: 1
Timestamp: 2026-05-04 08:44:31.159222
Data: Block 1 Data
Previous Hash: 98b43e28d1d88735e68435db6ba29d0df8c27edc1cdb33f61e07b0ac133f9c8f
Hash: 4013d800a046f4f137270c7e84e8d692eae99e041240620499ce174e6839cb64

Index: 2
Timestamp: 2026-05-04 08:44:31.159269
Data: Block 2 Data
Previous Hash: 4013d800a046f4f137270c7e84e8d692eae99e041240620499ce174e6839cb64
Hash: 7e96df07a71d88023dd06626e4f2dffccde39f9dbaa54674eb743eade0c71594

Index: 3
Timestamp: 2026-05-04 08:44:31.159286
Data: Block 3 Data
Previous Hash: 7e96df07a71d88023dd06626e4f2dffccde39f9dbaa54674eb743eade0c71594
Hash: 4cbb6fc61370df66bece9d464ce69cd1e5508ae9d57f0478876640c0001ec929
```

```
Index: 4
Timestamp: 2026-05-04 08:44:31.159298
Data: Block 4 Data
Previous Hash: 4cbb6fc61370df66bece9d464ce69cd1e5508ae9d57f0478876640c0001ec929
Hash: 9fb648ee6e322e7efedec1552f519f6b19bab763d2d1c440b0e6a4043e0da898
After Changing Block 2 Data :

Index: 0
Timestamp: 2026-05-04 08:44:31.158374
Data: Genesis Block
Previous Hash: 0
Hash: 98b43e28d1d88735e68435db6ba29d0df8c27edc1cdb33f61e07b0ac133f9c8f

Index: 1
Timestamp: 2026-05-04 08:44:31.159222
Data: Block 1 Data
Previous Hash: 98b43e28d1d88735e68435db6ba29d0df8c27edc1cdb33f61e07b0ac133f9c8f
Hash: 4013d800a046f4f137270c7e84e8d692eae99e041240620499ce174e6839cb64

Index: 2
Timestamp: 2026-05-04 08:44:31.159269
Data: Hacked Data
Previous Hash: 4013d800a046f4f137270c7e84e8d692eae99e041240620499ce174e6839cb64
Hash: 88f8427d573a00f57283d04659776530ed404fe060b4c7862bf4001bfddac3ed
```

```
Index: 3
Timestamp: 2026-05-04 08:44:31.159286
Data: Block 3 Data
Previous Hash: 7e96df07a71d88023dd06626e4f2dffccde39f9dbaa54674eb743eade0c71594
Hash: 4cbb6fc61370df66bece9d464ce69cd1e5508ae9d57f0478876640c0001ec929

Index: 4
Timestamp: 2026-05-04 08:44:31.159298
Data: Block 4 Data
Previous Hash: 4cbb6fc61370df66bece9d464ce69cd1e5508ae9d57f0478876640c0001ec929
Hash: 9fb648ee6e322e7efedec1552f519f6b19bab763d2d1c440b0e6a4043e0da898
```

Task 2:

Sol:

Input:

```python
p = 17
a = 2
b = 2

# Point Addition ----------------
def point_add(P, Q):
    if P == Q:
        return point_double(P)
    x1, y1 = P
    x2, y2 = Q

    m = ((y2 - y1) * pow(x2 - x1, -1, p)) % p
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p
    return (x3, y3)

# Point Doubling ----------------
def point_double(P):
    x, y = P
    m = ((3*x*x + a) * pow(2*y, -1, p)) % p
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p
    return (x3, y3)
```

```python
# Scalar Multiplication ----------------
def scalar_mult(k, P):
    R = P
    for i in range(k-1):
        R = point_add(R, P)
    return R

# Key Generation ----------------
G = (5, 1)   # Generator point
private_key = 3
public_key = scalar_mult(private_key, G)

print("Private Key:", private_key)
print("Public Key:", public_key)

# Simple Encryption ----------------
message = 9   # numeric message

k = 2
C1 = scalar_mult(k, G)
C2 = message + scalar_mult(k, public_key)[0]
print("Cipher:", (C1, C2))

# Decryption ----------------
shared = scalar_mult(private_key, C1)
decrypted = C2 - shared[0]
print("Decrypted Message:", decrypted)
```

Output:

```
t_03.py"
Private Key: 3
Public Key: (10, 6)
Cipher: ((6, 3), 25)
Decrypted Message: 9
```

Task 1 Explanation:

- Each block contains index, timestamp, data, previous hash, and its own hash. The hash is created using SHA256.

- Each block stores the previous block's hash, which links all blocks together.

- If someone changes the data in a block, its hash changes. This breaks the chain and makes the blockchain invalid and insecure.

Task 2 Explanation:

- ECC is based on a mathematical curve:
  $y^2 = x^3 + ax + b \pmod p$

- Point addition means adding two different points.
  while
  Point doubling means adding the same point to itself.

- Private key is a secret number.
  while
  Public key is calculated by multiplying the private key with a generator point.

- For encryption, a random number k is chosen.
  $C1 = k \times G$
  $C2 = \text{message} + (k \times \text{public key})$

- For decryption:
  $\text{shared} = \text{private key} \times C1$
  $\text{message} = C2 - \text{shared}$

Security Analysis:

Blockchain	ECC
• **Blockchain is immutable, meaning changes can be detected.**	• ECC is more secure than RSA with smaller key sizes.

• **Tampering is easy to detect.**	• It is faster and more efficient.
• **Real blockchains are distributed.**	• It is used in real systems like Bitcoin and SSL.